Segment Trees: Algorithms and a Robust Python Implementation

Scope and conventions

This document presents a segment tree for range-sum queries with point insertion, point deletion, and traversal. The tree has a fixed index domain: the array has length n, and operations do not shift later indices. “Insert” activates a previously inactive position; “delete” deactivates an active position. An inactive position contributes zero to sums, but is tracked separately so an active value of zero remains valid.

The implementation uses zero-based Python indices and a recursive tree stored in arrays. For n > 0, an allocation of 4n tree entries is sufficient. The empty-array case n = 0 is handled explicitly.

Core invariants

• A leaf represents exactly one index.

• An active leaf stores its value; an inactive leaf contributes zero.

• An internal node stores the sum of its two children.

• Query bounds are inclusive and zero-based.

Reference pseudocode

Point insertion

```
INSERT(v, l, r, p, x)
    if l = r:
        mark p active
        sum[v] = x
        return
    mid = floor((l + r) / 2)
    recurse into the child containing p
    sum[v] = sum[left[v]] + sum[right[v]]
```

Point deletion

```
DELETE(v, l, r, p)
    if l = r:
        if p is inactive: return
        mark p inactive
        sum[v] = 0
        return
    mid = floor((l + r) / 2)
    recurse into the child containing p
    sum[v] = sum[left[v]] + sum[right[v]]
```

Inorder traversal

```
TRAVERSE(v, l, r)
    if l = r:
        if l is active: print its value
        return
    mid = floor((l + r) / 2)
    TRAVERSE(left[v], l, mid)
    print sum[v]              # internal aggregate
    TRAVERSE(right[v], mid + 1, r)
```

Range-sum query

```
RANGE-QUERY(v, l, r, i, j)
    if r < i or j < l: return 0
    if i <= l and r <= j: return sum[v]
    mid = floor((l + r) / 2)
    return RANGE-QUERY(left[v], l, mid, i, j)
         + RANGE-QUERY(right[v], mid + 1, r, i, j)
```

To print only array values in index order, omit the internal-node print statement.

Robust Python implementation

Common bug fixes include validation of indices and ranges; handling empty input; tracking activity separately from values so zero is valid; rejecting duplicate insertion; making deletion of an inactive position a no-op; recomputing ancestors after updates; using zero for disjoint sum-query intervals; and distinguishing leaf values from internal

aggregates.

```
class SegmentTree:
    """Fixed-length segment tree for point activation/deactivation and range sums."""
    def __init__(self, values):
        self.n = len(values)
        self.values = list(values)
        self.active = [True] * self.n
        self.tree = [0] * (4 * self.n if self.n else 0)
        if self.n:
            self._build(1, 0, self.n - 1)

    def _build(self, node, left, right):
        if left == right:
            self.tree[node] = self.values[left]
            return
        mid = (left + right) // 2
        self._build(node * 2, left, mid)
        self._build(node * 2 + 1, mid + 1, right)
        self._pull(node)

    def _pull(self, node):
        self.tree[node] = self.tree[node * 2] + self.tree[node * 2 + 1]

    def _check_index(self, index):
        if not isinstance(index, int) or isinstance(index, bool):
            raise TypeError("index must be an integer")
        if not 0 <= index < self.n:
            raise IndexError("index out of range")

    def _check_range(self, left, right):
        for name, value in (("left", left), ("right", right)):
            if not isinstance(value, int) or isinstance(value, bool):
                raise TypeError(f"{name} must be an integer")
        if not (0 <= left <= right < self.n):
            raise IndexError("require 0 <= left <= right < n")

    def insert(self, index, value):
        """Activate an inactive position; raise if it is already active."""
        self._check_index(index)
        if self.active[index]:
            raise ValueError("index is already active")
        self.values[index] = value
        self.active[index] = True
        self._update(1, 0, self.n - 1, index, value)

    def delete(self, index):
        """Deactivate a position; deleting an inactive position is a no-op."""
        self._check_index(index)
        if not self.active[index]:
            return
        self.active[index] = False
        self._update(1, 0, self.n - 1, index, 0)

    def update(self, index, value):
        """Replace the value at an active position."""
        self._check_index(index)
        if not self.active[index]:
            raise ValueError("cannot update an inactive index; use insert")
        self.values[index] = value
        self._update(1, 0, self.n - 1, index, value)

    def _update(self, node, left, right, index, value):
        if left == right:
            self.tree[node] = value
            return
        mid = (left + right) // 2
        if index <= mid:
            self._update(node * 2, left, mid, index, value)
        else:
            self._update(node * 2 + 1, mid + 1, right, index, value)
        self._pull(node)

    def range_sum(self, left, right):
        """Sum over the inclusive interval [left, right]."""
        self._check_range(left, right)
        return self._query(1, 0, self.n - 1, left, right)

    def _query(self, node, seg_left, seg_right, q_left, q_right):
        if q_right < seg_left or seg_right < q_left:
            return 0
        if q_left <= seg_left and seg_right <= q_right:
            return self.tree[node]
        mid = (seg_left + seg_right) // 2
        return (
            self._query(node * 2, seg_left, mid, q_left, q_right)
            + self._query(node * 2 + 1, mid + 1, seg_right, q_left, q_right)
        )

    def traverse_inorder(self, include_internal_sums=False):
```

```
        """Return active values in index order, or tagged inorder tree events."""
        if not self.n:
            return []
        if not include_internal_sums:
            return [self.values[i] for i in range(self.n) if self.active[i]]
        events = []
        def visit(node, left, right):
            if left == right:
                if self.active[left]:
                    events.append(("value", left, self.values[left]))
                return
            mid = (left + right) // 2
            visit(node * 2, left, mid)
            events.append(("sum", left, right, self.tree[node]))
            visit(node * 2 + 1, mid + 1, right)
        visit(1, 0, self.n - 1)
        return events

    def total(self):
        """Sum of active values, or zero for an empty tree."""
        return self.tree[1] if self.n else 0
```

Example use

```
st = SegmentTree([2, 5, 1, 4, 3])
assert st.range_sum(1, 3) == 10
st.delete(2)
assert st.range_sum(1, 3) == 9
st.insert(2, 7)
assert st.range_sum(1, 3) == 16
st.update(1, 6)
assert st.range_sum(0, 4) == 22
assert st.traverse_inorder() == [2, 6, 7, 4, 3]

empty = SegmentTree([])
assert empty.total() == 0
assert empty.traverse_inorder() == []
```

Complexity

Operation	Time
Build	O(n)
Insert / delete / update	O(log n)
Range-sum query	O(log n)
Full traversal	O(n)
Space	O(n)

Important design boundary

This is a fixed-index segment tree. Its insert and delete methods activate and deactivate positions; they do not change sequence length or shift indices. True sequence insertion/deletion requires a dynamic sequence structure, such as an implicit treap, or a segment tree designed over an expandable domain.

References

Jeff Erickson, *Algorithms*, author-hosted materials: https://jeffe.cs.illinois.edu/teaching/algorithms/. For a direct implementation reference, see CP-Algorithms, “Segment Tree”: https://cp-algorithms.com/data_structures/segment_tree.html. The code here is self-contained, not copied verbatim from either source.